Verano, Kierstien S. BSIT-3

Capture the Flag 8: Enroll a New Student

Goal: Extend your multi-screen Student Directory by implementing student enrollment using form fields, controllers, validation, submission, and local data persistence while preserving your existing navigation, student details, and edit modal.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does.
- Understand syntax.
- Explain an error (not fix your error).
- Clarify documentation.

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before implementing student enrollment.

Create:

`feature/student-enrollment`

Screenshot:

- Terminal showing your current branch.

```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/multi-scre
en-navigation)
$ git status
On branch feature/multi-screen-navigation
Your branch is up to date with 'origin/feature/multi-screen-navigation'.

nothing to commit, working tree clean

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/multi-scre
en-navigation)
$ git checkout -b feature/student-enrollment
Switched to a new branch 'feature/student-enrollment'

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-en
rollment)
$ git branch
  feature/conditional-ui
  feature/data-ui
  feature/exam
  feature/exam_2
  feature/midterm_exam
  feature/multi-screen-navigation
* feature/student-enrollment
  feature/student-interactions
  feature/student-list
  feature/widgets
  main
```

🏴 Flag 1 — Build the Add Student Form

Objective: Transform your existing Add Student Screen into a functional enrollment form.

Add the following input fields:

- Student ID
- Full Name
- Course
- Year Level
- Email Address

Research how Flutter uses:

- `TextField` or `TextFormField`
- `TextEditingController`
- Form layouts and input fields

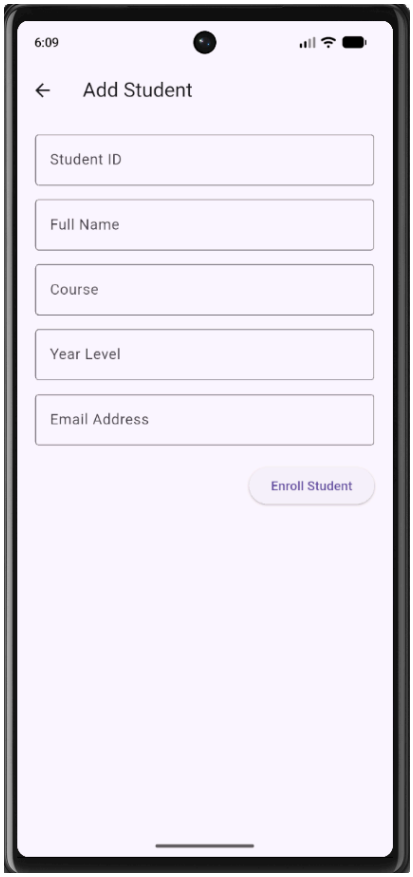
Test

The Add Student Screen should display all required fields and allow administrators to enter student information.

The screen must remain accessible through your existing Student List navigation.

Screenshot:

- Add Student Screen with all input fields.
- Controllers declared in your code.
- Application displaying the enrollment form.




```dart
class _AddStudentScreenState extends State<AddStudentScreen> {
  final studentIdController = TextEditingController();
  final fullNameController = TextEditingController();
  final courseController = TextEditingController();
  final yearLevelController = TextEditingController();
  final emailController = TextEditingController();
```

🏴 Flag 2 — Validate Student Information

Objective: Prevent incomplete, invalid, or duplicate student information from being submitted.

Implement form validation with the following requirements:

- All required fields must be filled in.
- Student ID must not duplicate an existing student's ID.
- Email Address must follow a valid email format.
- Year Level must contain an appropriate year level value.

Research how Flutter handles form validation and displays validation messages.

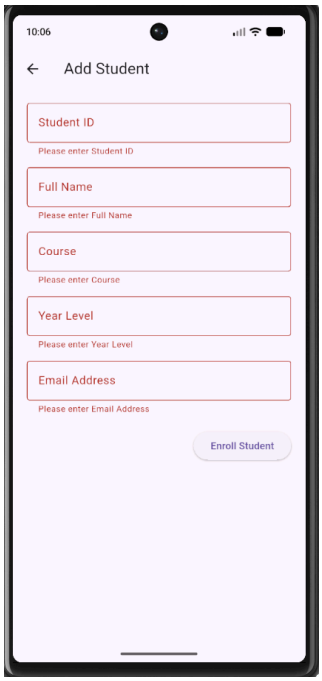
Test

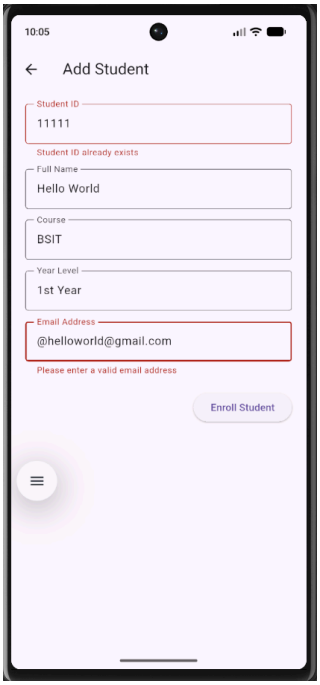
1. Submit the form with all fields empty.
2. Enter an invalid email address.
3. Enter a Student ID that already exists.
4. Enter valid information into every field.

Invalid information should display appropriate validation messages and must not create a student record.

Screenshot:

- Required-field validation.
- Invalid email validation.
- Duplicate Student ID validation.
- Validation logic in your code.






```
49                validator: (value) {
50                  if (value == null || value.trim().isEmpty) {
51                    return "Please enter Student ID";
52                  }
53
54                  if (profiles.any(
55                    (profile) => profile.studentId.trim() == value.trim(),
56                  )) {
57                    return "Student ID already exists";
58                  }
```

```
72                validator: (value) {
73                  if (value == null || value.trim().isEmpty) {
74                    return "Please enter Full Name";
75                  }
```

```
89                validator: (value) {
90                  if (value == null || value.trim().isEmpty) {
91                    return "Please enter Course";
92                  }
```

```
106               validator: (value) {
107                 if (value == null || value.trim().isEmpty) {
108                   return "Please enter Year Level";
109                 }
110
111                 if (![
112                   "1st year",
113                   "2nd year",
114                   "3rd year",
115                   "4th year",
116                 ].contains(value.trim().toLowerCase())) {
117                   return "Enter 1st, 2nd, 3rd, or 4th Year";
118                 }
```

```
133               validator: (value) {
134                 if (value == null || value.trim().isEmpty) {
135                   return "Please enter Email Address";
136                 }
137
138                 if (value.startsWith("@") ||
139                     value.endsWith(".") ||
140                     value.contains(" ") ||
141                     !value.contains("@") ||
142                     !value.contains(".")) {
143                   return "Please enter a valid email address";
144                 }
```

🏴 Flag 3 — Enroll and Display a New Student

Objective: Add a successfully enrolled student to your existing Student Directory.

When valid information is submitted:

- Create a new student record.
- Add the student to your existing student list.
- Return to the Student List Screen or update it appropriately.
- Display the newly enrolled student.
- Clear the form fields after successful submission.

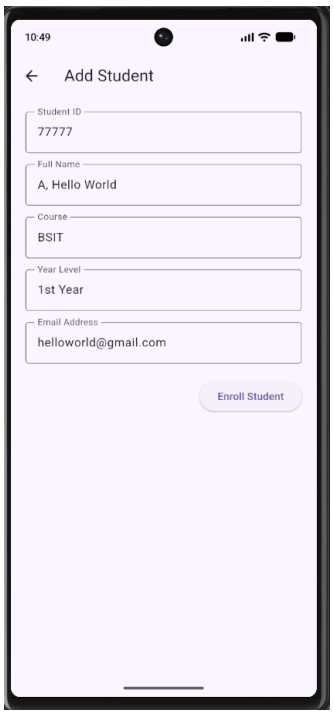
Research how Flutter handles form submission, updating application state, and returning from a screen using navigation.

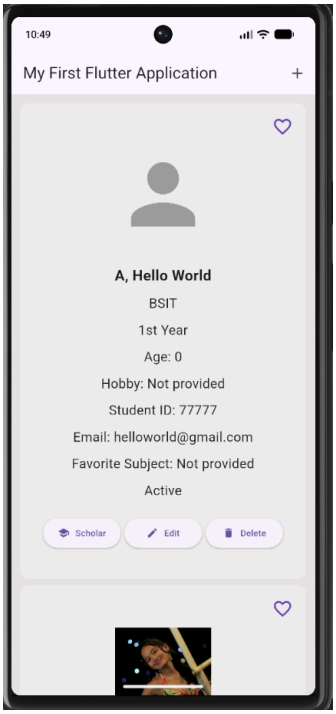
Test

1. Open the Add Student Screen.
2. Enter valid student information.
3. Submit the form.
4. Verify that the new student appears in the Student List.
5. Select the new student.
6. Verify that the Student Details Screen displays the correct information.

Screenshot:

- Completed enrollment form.
- Newly enrolled student in the Student List.
- Student Details Screen displaying the new student's information.
- Submission and navigation logic.






```
158                 Profile newStudent = Profile(
159                   image: "",
160                   name: fullNameController.text.trim(),
161                   course: courseController.text.trim(),
162                   yearLevel: yearLevelController.text.trim(),
163                   age: 0,
164                   hobby: "Not provided",
165                   studentId: studentIdController.text.trim(),
166                   email: emailController.text.trim(),
167                   favoriteSubject: "Not provided",
168                   active: true,
169                 ); // Profile
170
171                 profiles.add(newStudent);
172
173                 profiles.sort((a, b) => a.name.compareTo(b.name));
174
175                 studentIdController.clear();
176                 fullNameController.clear();
177                 courseController.clear();
178                 yearLevelController.clear();
179                 emailController.clear();
180
181                 Navigator.pop(context, true);
```

```
326             onPressed: () async {
327               final result = await Navigator.pushNamed(context, '/add-student');
328
329               if (result == true) {
330                 setState(() {});
331               }
332             },
```

🏴 Flag 4 — Save Student Records Locally

Objective: Preserve student records even after the application is closed and restarted.

Research Flutter-compatible local data persistence options and choose an appropriate method.

Your implementation must:

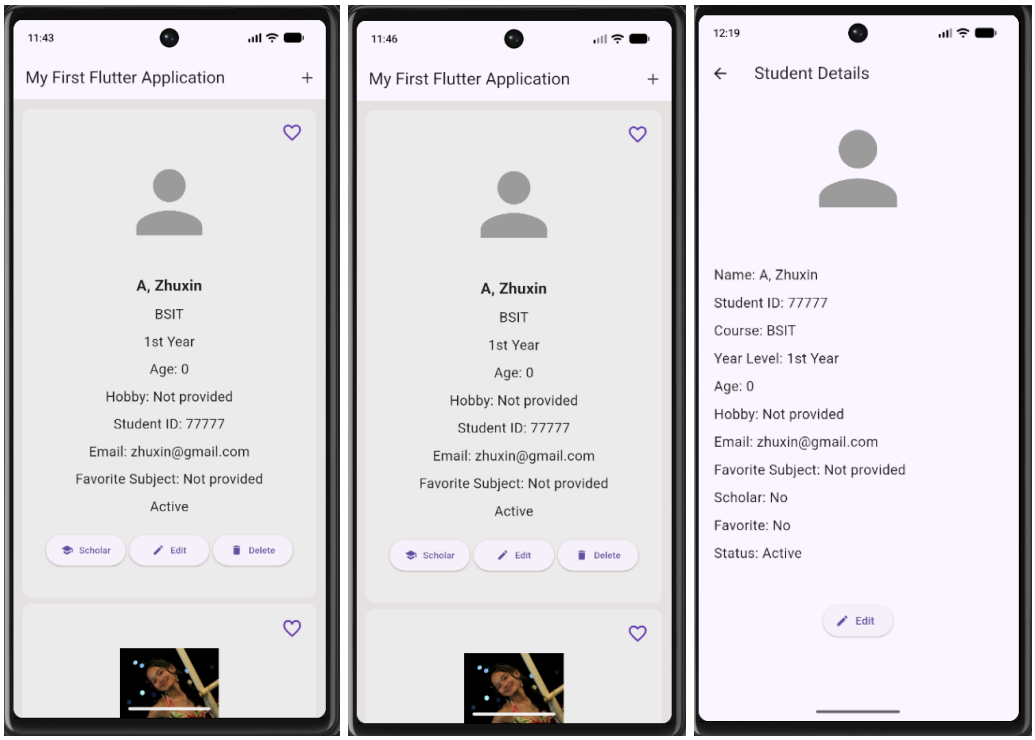
- Save student records locally.
- Load saved records when the application starts.
- Preserve newly enrolled students after restarting the application.
- Handle situations where no saved records exist yet.

Test

1. Enroll a new student.
2. Close and restart the application.
3. Verify that the newly enrolled student still appears in the Student List.
4. Open the student's details and verify that the information remains correct.

Screenshot:

- Student List displaying saved records.
- Local persistence implementation.
- Student Details Screen displaying a saved student after restarting the application.



🏴 Flag 5 — Keep Existing Features Functional

Objective: Ensure that enrollment works with the navigation and interactions implemented in CTF 7.

Your application must retain:

- Student List Screen.
- Student Details Screen.
- Student information passed between screens.
- Edit modal.
- Add Student Screen.
- Existing named routes.

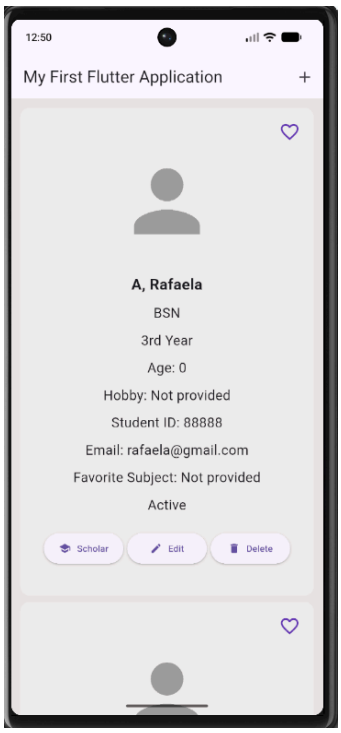
The enrollment workflow must integrate with these existing features rather than replacing them with a single screen.

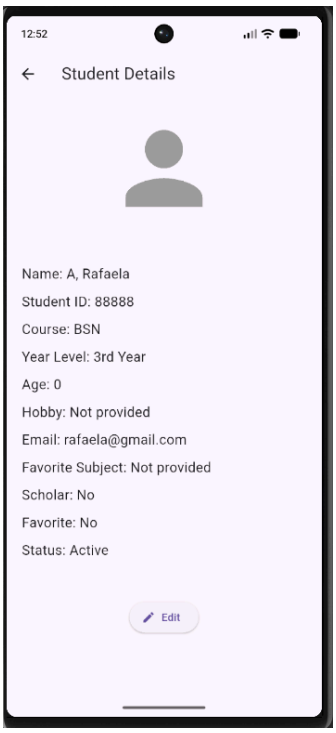
Test

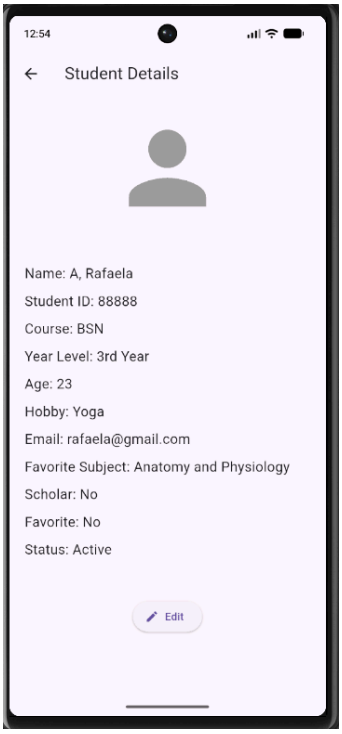
1. Navigate from Student List to Add Student.
2. Enroll a new student.
3. Open the newly enrolled student's details.
4. Open the Edit modal.
5. Modify the student's information.
6. Close the modal and verify the updated information.
7. Navigate back to the Student List.
8. Verify that the student information remains consistent.

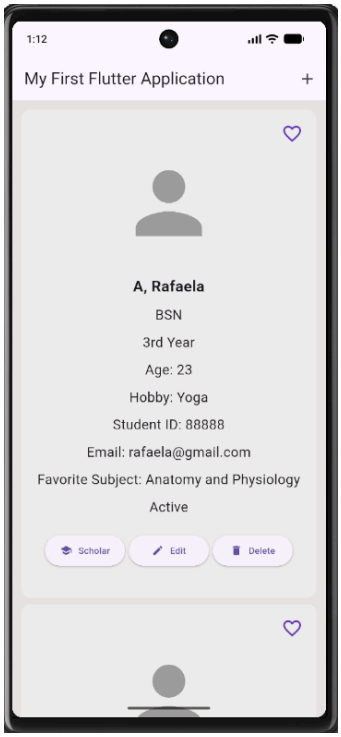
Screenshot:

- Student List with the newly enrolled student.
- Student Details displaying the correct information.
- Edit modal functioning correctly.
- Existing navigation working with the enrollment feature.









PS: I'll add the change photo feature later on miss hehe. I'll just focus first on what's asked and needed for now.

🏴 Flag 6 — Synchronize Changes with Local Data

Objective: Ensure that creating, editing, and deleting students updates the locally saved records.

Your application should keep its student list and locally persisted data consistent.

Verify that:

- Newly enrolled students are saved.
- Edited student information is saved.
- Deleted student records remain deleted after restarting.
- Student IDs remain unique.
- The Student List reflects the latest student records.

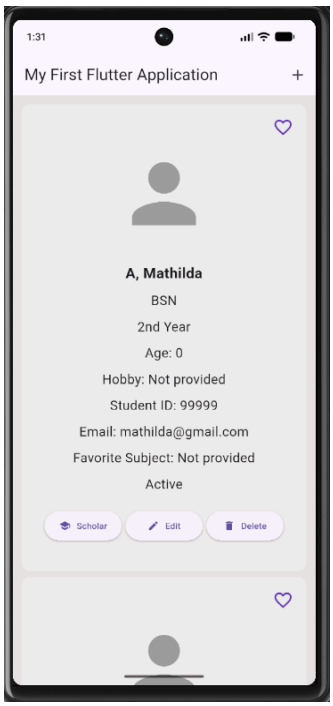
Research how to update and remove persisted records using your chosen local storage method.

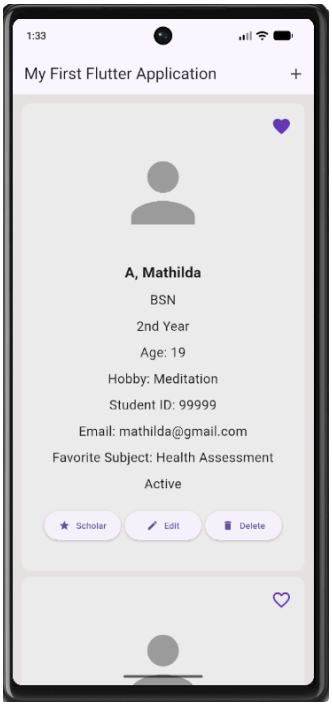
Test

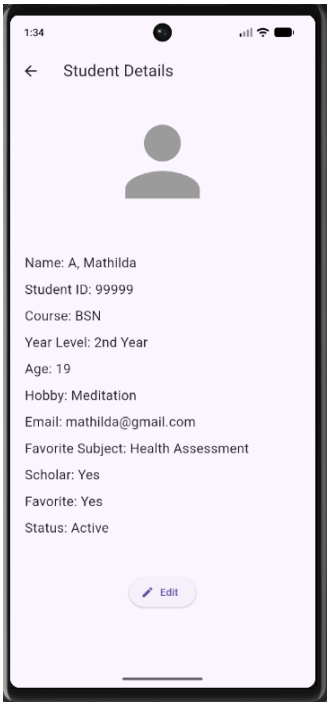
1. Enroll a new student.
2. Edit the student's information using the modal.
3. Delete a student using your existing delete functionality, if implemented.
4. Restart the application.
5. Verify that the saved records reflect the changes.

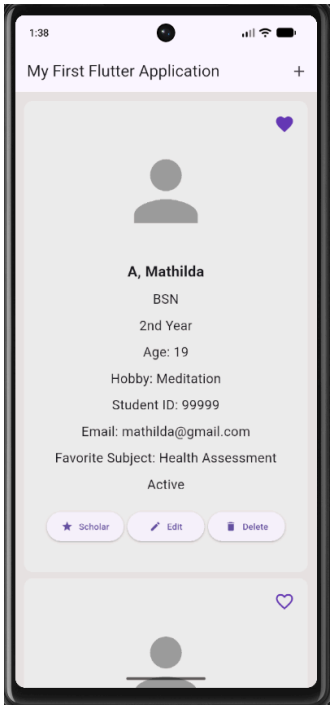
Screenshot:

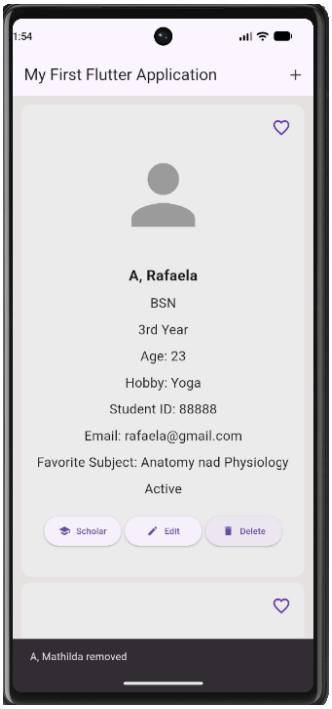
- Student List after enrollment and editing.
- Updated student details.
- Student records after restarting the application.
- Persistence logic for creating, updating, and deleting records.

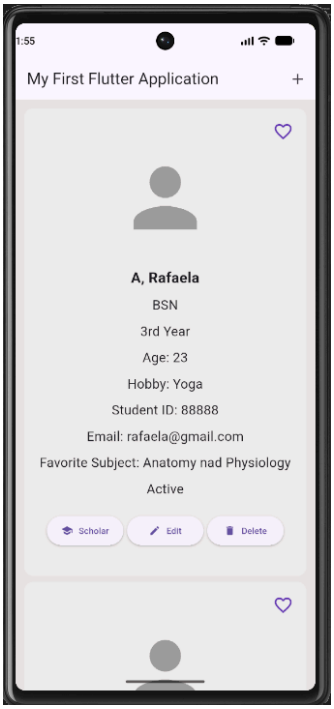

1:31
My First Flutter Application
A, Mathilda
BSN
2nd Year
Age: 0
Hobby: Not provided
Student ID: 99999
Email: mathilda@gmail.com
Favorite Subject: Not provided
Active
Scholar
Edit
Delete


1:33
My First Flutter Application
A, Mathilda
BSN
2nd Year
Age: 19
Hobby: Meditation
Student ID: 99999
Email: mathilda@gmail.com
Favorite Subject: Health Assessment
Active
Scholar
Edit
Delete


1:34
Student Details
Name: A, Mathilda
Student ID: 99999
Course: BSN
Year Level: 2nd Year
Age: 19
Hobby: Meditation
Email: mathilda@gmail.com
Favorite Subject: Health Assessment
Scholar: Yes
Favorite: Yes
Status: Active
Edit


1:38
My First Flutter Application
A, Mathilda
BSN
2nd Year
Age: 19
Hobby: Meditation
Student ID: 99999
Email: mathilda@gmail.com
Favorite Subject: Health Assessment
Active
Scholar
Edit
Delete


1:54
My First Flutter Application
A, Rafaela
BSN
3rd Year
Age: 23
Hobby: Yoga
Student ID: 88888
Email: rafaela@gmail.com
Favorite Subject: Anatomy nad Physiology
Active
Scholar
Edit
Delete
A, Mathilda removed


1:55
My First Flutter Application
A, Rafaela
BSN
3rd Year
Age: 23
Hobby: Yoga
Student ID: 88888
Email: rafaela@gmail.com
Favorite Subject: Anatomy nad Physiology
Active
Scholar
Edit
Delete

🏴 Flag 7 — Verify the Complete Enrollment Workflow

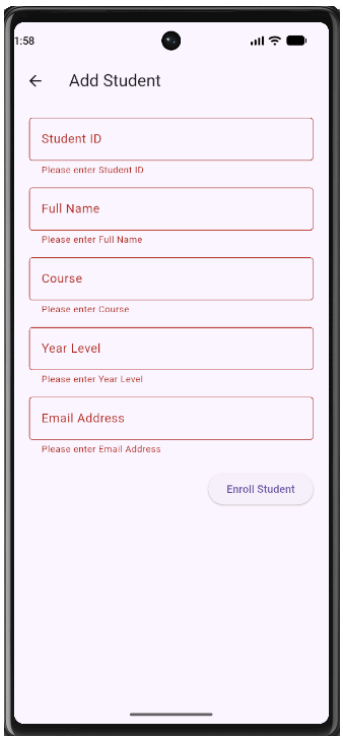
Objective: Demonstrate that the Student Directory supports enrollment, validation, navigation, editing, and local data persistence.

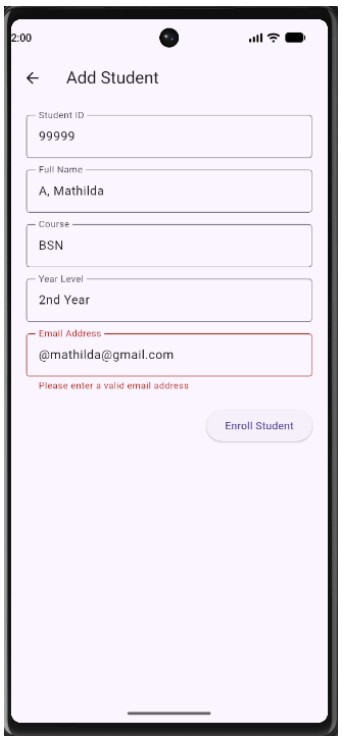
Test Scenario

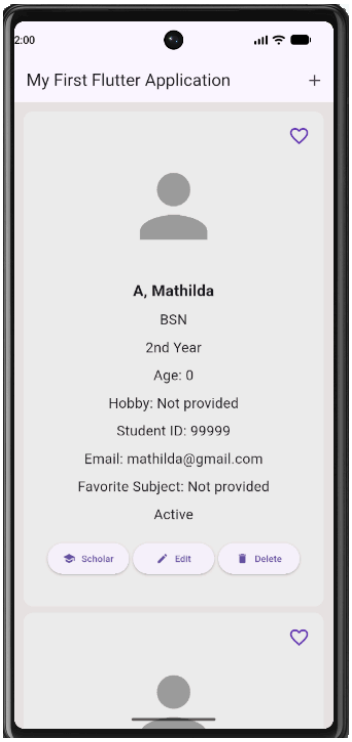
1. Launch the application.
2. Verify that saved student records load.
3. Navigate to Add Student.
4. Submit an empty form.
5. Submit an invalid email address.
6. Attempt to enroll a student with a duplicate Student ID.
7. Enter valid student information.
8. Submit the form successfully.
9. Verify that the student appears in the Student List.
10. Open Student Details and verify the student's information.
11. Edit the student using the modal.
12. Restart the application.
13. Verify that the enrolled student and updated information remain available.

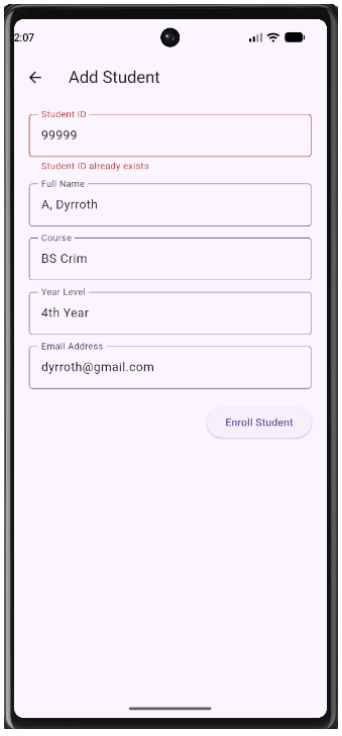
Screenshot:

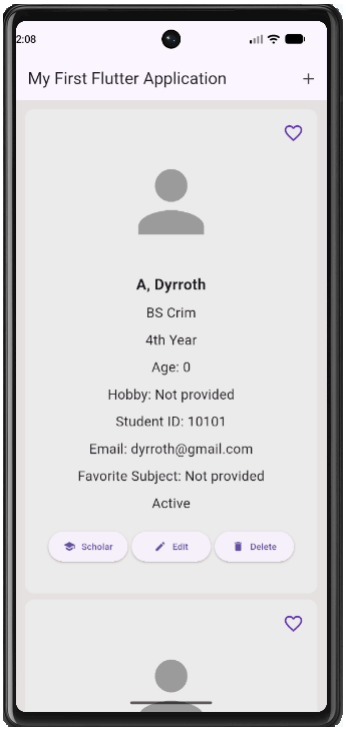
- Enrollment form with validation errors.
- Successful enrollment.
- Student List displaying the new student.
- Student Details and Edit modal.
- Saved student information after restarting the application.

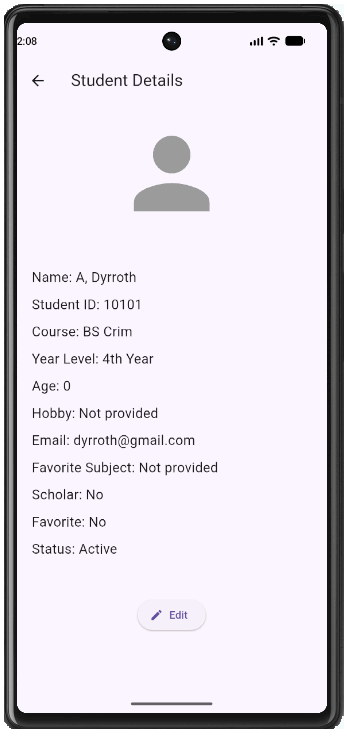

1:58
Add Student
Student ID
Please enter Student ID
Full Name
Please enter Full Name
Course
Please enter Course
Year Level
Please enter Year Level
Email Address
Please enter Email Address
Enroll Student

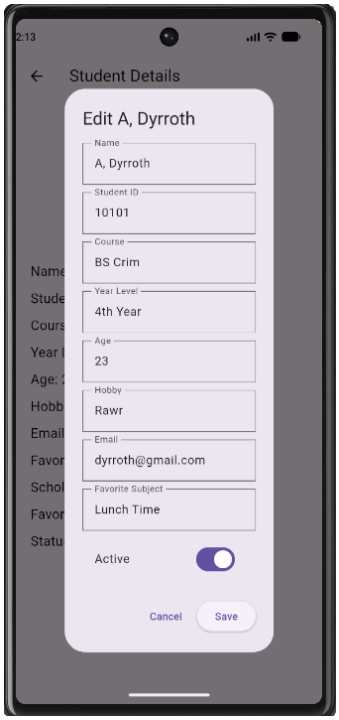

2:00
Add Student
Student ID
99999
Full Name
A, Mathilda
Course
BSN
Year Level
2nd Year
Email Address
@mathilda@gmail.com
Please enter a valid email address
Enroll Student

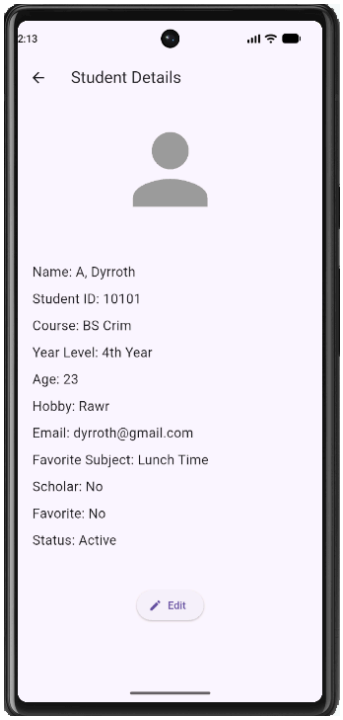

2:00
My First Flutter Application
A, Mathilda
BSN
2nd Year
Age: 0
Hobby: Not provided
Student ID: 99999
Email: mathilda@gmail.com
Favorite Subject: Not provided
Active
Scholar
Edit
Delete

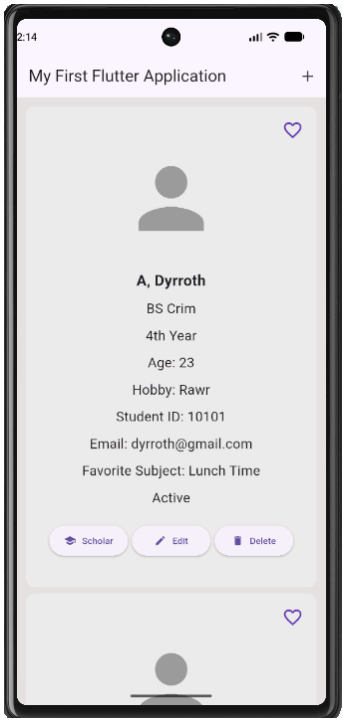

2:07
Add Student
Student ID
99999
Student ID already exists
Full Name
A, Dyrroth
Course
BS Crim
Year Level
4th Year
Email Address
dyrroth@gmail.com
Enroll Student

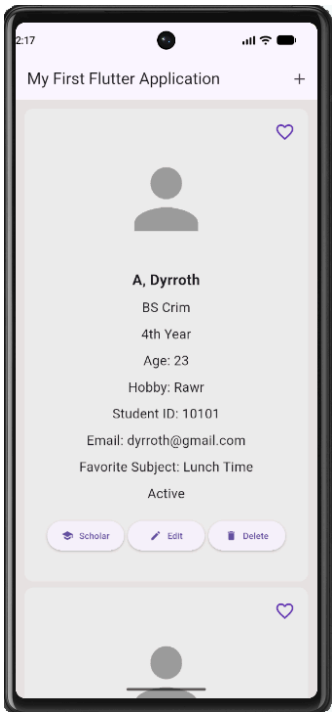

2:08
My First Flutter Application
A, Dyrroth
BS Crim
4th Year
Age: 0
Hobby: Not provided
Student ID: 10101
Email: dyrroth@gmail.com
Favorite Subject: Not provided
Active
Scholar
Edit
Delete


2:08
Student Details
Name: A, Dyrroth
Student ID: 10101
Course: BS Crim
Year Level: 4th Year
Age: 0
Hobby: Not provided
Email: dyrroth@gmail.com
Favorite Subject: Not provided
Scholar: No
Favorite: No
Status: Active
Edit


2:13
Student Details
Edit A, Dyrroth
Name
A, Dyrroth
Student ID
10101
Course
BS Crim
Year Level
4th Year
Age
23
Hobby
Rawr
Email
dyrroth@gmail.com
Favorite Subject
Lunch Time
Active
Cancel
Save


2:13
Student Details
Name: A, Dyrroth
Student ID: 10101
Course: BS Crim
Year Level: 4th Year
Age: 23
Hobby: Rawr
Email: dyrroth@gmail.com
Favorite Subject: Lunch Time
Scholar: No
Favorite: No
Status: Active
Edit


2:14
My First Flutter Application
A, Dyrroth
BS Crim
4th Year
Age: 23
Hobby: Rawr
Student ID: 10101
Email: dyrroth@gmail.com
Favorite Subject: Lunch Time
Active
Scholar
Edit
Delete


2:17
My First Flutter Application
A, Dyrroth
BS Crim
4th Year
Age: 23
Hobby: Rawr
Student ID: 10101
Email: dyrroth@gmail.com
Favorite Subject: Lunch Time
Active
Scholar
Edit
Delete

🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student Enrollment feature.

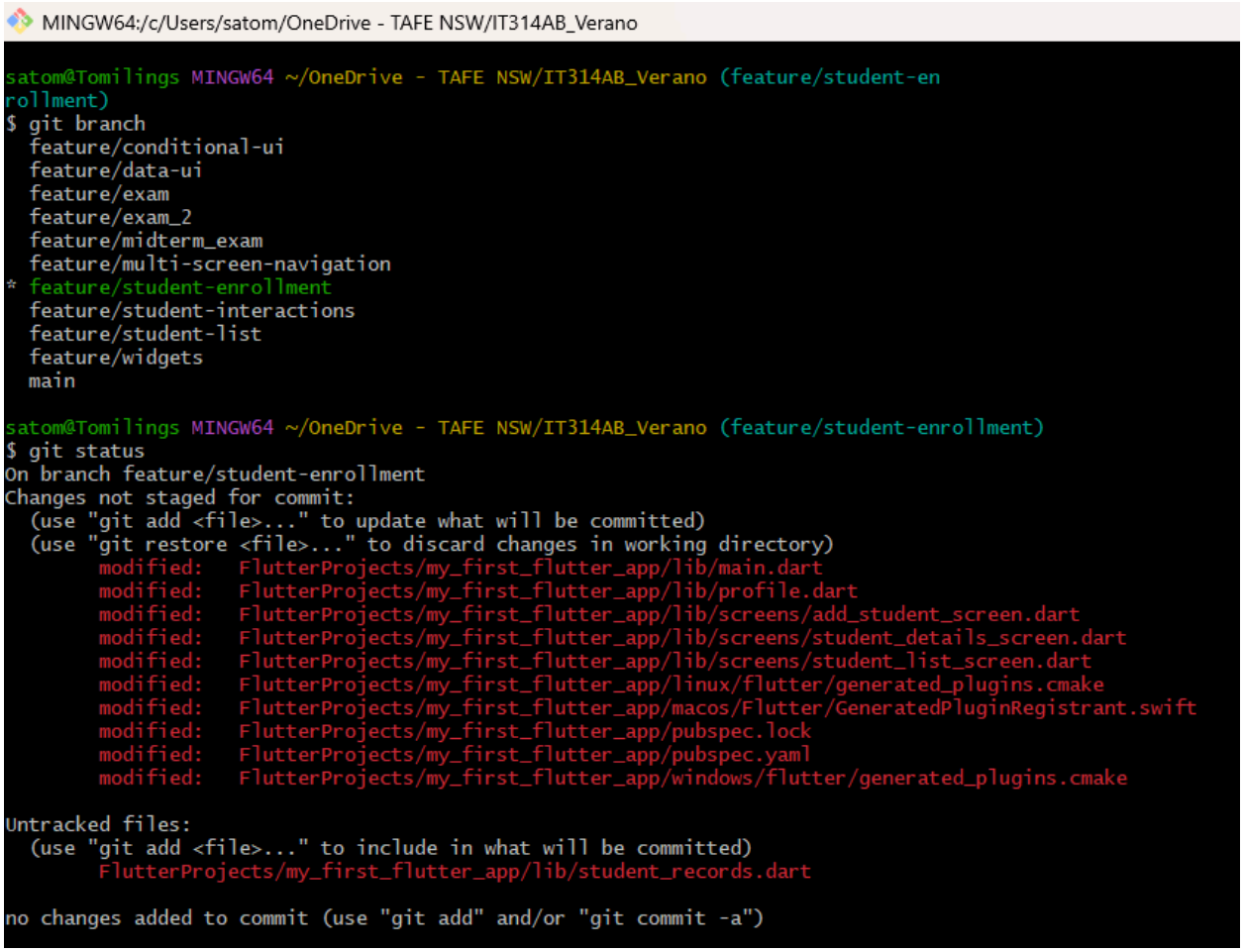
Use a proper Git commit message.

Branch:

`feature/student-enrollment`

Screenshot:

- Successful commit.
- Current branch.
- Terminal showing the commit.
- Successful push to the remote repository.



```
satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-enrollment)
$ git add .
warning: in the working copy of 'FlutterProjects/my_first_flutter_app/linux/flutter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'FlutterProjects/my_first_flutter_app/macos/Flutter/GeneratedPluginRegistrant.swift', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'FlutterProjects/my_first_flutter_app/windows/flutter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touches it

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-enrollment)
$ git status
On branch feature/student-enrollment
Your branch is up to date with 'origin/feature/student-enrollment'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   FlutterProjects/my_first_flutter_app/lib/main.dart
        modified:   FlutterProjects/my_first_flutter_app/lib/profile.dart
        modified:   FlutterProjects/my_first_flutter_app/lib/screens/add_student_screen.dart
        modified:   FlutterProjects/my_first_flutter_app/lib/screens/student_details_screen.dart
        modified:   FlutterProjects/my_first_flutter_app/lib/screens/student_list_screen.dart
        new file:   FlutterProjects/my_first_flutter_app/lib/student_records.dart
        modified:   FlutterProjects/my_first_flutter_app/linux/flutter/generated_plugins.cmake
        modified:   FlutterProjects/my_first_flutter_app/macos/Flutter/GeneratedPluginRegistrant.swift
        modified:   FlutterProjects/my_first_flutter_app/pubspec.lock
        modified:   FlutterProjects/my_first_flutter_app/pubspec.yaml
        modified:   FlutterProjects/my_first_flutter_app/windows/flutter/generated_plugins.cmake


satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-enrollment)
$ git commit -m "feat: submit Enroll a New Student"
[feature/student-enrollment f4836a6] feat: submit Enroll a New Student
 11 files changed, 682 insertions(+), 37 deletions(-)
 create mode 100644 FlutterProjects/my_first_flutter_app/lib/student_records.dart

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-enrollment)
$ git status
On branch feature/student-enrollment
Your branch is ahead of 'origin/feature/student-enrollment' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

satom@Tomilings MINGW64 ~/OneDrive - TAFE NSW/IT314AB_Verano (feature/student-enrollment)
$ git push
Enumerating objects: 43, done.
Counting objects: 100% (43/43), done.
Delta compression using up to 20 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (23/23), 7.61 KiB | 2.54 MiB/s, done.
Total 23 (delta 15), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (15/15), completed with 12 local objects.
To https://github.com/KVerano/IT314AB_Verano.git
   aa56c63..f4836a6  feature/student-enrollment -> feature/student-enrollment
```